Disciplina: Sistemas Distribuídos
Professora: Ana Cristina Barreiras Kochem Vendramin

Avaliação (valor 2,5)
Microsserviços, Sistema de Mensageria, REST, SSE

Evoluir a arquitetura do e-commerce do **Trab1** para uma aplicação Web completa baseada em microsserviços. O projeto combina comunicação **REST** entre *Frontend* e API *Gateway*, notificações em tempo real via **SSE**, mensageria com **RabbitMQ** e integrações com **APIs Externas (E-mail e Pagamento via Webhook)**.

Visão Geral da Arquitetura

1. ***Frontend* web:** interface gráfica desenvolvida em linguagem **diferente do *backend***. O *frontend* comunicará com o *backend* via REST.
2. **MS Principal: ponto de entrada (API *Gateway*)** consumido pelo *Frontend* para acessar os serviços do *Backend*. **Gerencia as conexões SSE (*Server-Sent Events*), sendo responsável pelo envio das notificações em tempo real,** e traduz as requisições em eventos para o RabbitMQ.
3. RabbitMQ: responsável pela comunicação assíncrona entre os microsserviços do *backend*.
4. **Mock de Pagamento:** sistema externo acionado por URL em nova aba do navegador. Processa a escolha de pagamento do usuário e envia a resposta para o **MS Pagamento** via **Webhook**.
5. **API Externa de E-mail:** envio de e-mails para informar promoções aos consumidores cadastrados.
6. Assinatura Digital: criptografia assimétrica para garantir autenticidade e integridade em todas as mensagens trafegadas no RabbitMQ.

Detalhamento dos Componentes (ver Figura 1)

Frontend web (Valor: 0,5)

- **Catálogo e Pedidos:** consulta os produtos disponíveis e realiza pedidos via REST.
- **Pagamento:** ao finalizar o pedido, recebe a URL do Mock de Pagamento e abre uma nova aba no navegador para simular o pagamento.
- **Acompanhamento de Pedidos (SSE):** mantém conexão SSE com o API *gateway* para receber notificações em tempo real das alterações do status do pedido (*Estoque Confirmado, Pagamento Aprovado/Recusado, Pedido Enviado*).
- **Inscrição de Promoções:** permite ao consumidor cadastrar seu e-mail e selecionar as **categorias de interesse** para receber **ofertas por e-mail**.

Microsserviços *Backend* (Valor: 1,8)

MS Principal / API Gateway (valor: 1,0)

Responsável por:

- **expor a API REST consumida pelo *frontend*;**
- transformar ações dos usuários em eventos publicados no RabbitMQ;
- **(0,1) publicar os eventos interesse.promocao**, pedido.criado, pedido.excluído.
- Consumir todos os eventos gerados pelo microsserviços Estoque, Pagamento e Entrega (pedido.estoque_ok, estoque.indisponivel, pagamento.aprovado, pagamento.recusado, pedido.enviado)
- **(0,5) manter conexões SSE com o *frontend*** para envio de notificações **aos clientes sobre o status do(s) seu(s) pedido(s). As notificações SSE devem incluir toda mudança de estado sobre o pedido: *Estoque Confirmado, Pagamento Aprovado/Recusado, Pedido Enviado.***

O API Gateway deve disponibilizar *endpoints* REST para:

- **(0,1) listar produtos disponíveis em estoque;**
- **(0,1) criar pedidos;**
- **(0,1) registrar interesse, informando o e-mail, em receber notificação sobre promoções de categorias;**
- **(0,1) cancelar interesse em receber e-mail sobre.**

Obs.: O API Gateway deve **consultar a lista de produtos diretamente do MS Estoque (via REST)**.

MS Estoque (Valor: 0,2)

- Consome pedido.criado e pedido.excluido.
- Verifica a disponibilidade dos produtos, realiza a reserva/baixa e publica pedido.estoque_ok ou estoque.indisponivel.
- **(0,1) Persiste os dados dos produtos em estoque**.
- **(0,1) Expõe um *endpoint* para o MS Gateway consultar produtos disponíveis.**

MS Pagamento (valor 0,2)

- Consome pedido.estoque_ok.
- **(0,1) Solicita a criação de uma cobrança ao Mock de Pagamento para este gerar a URL de *checkout*, passando a sua URL de *Webhook* para retorno.**
- **(0,1) Disponibiliza o *endpoint* HTTP para receber o *Webhook* do Mock de Pagamento com o status APROVADO ou RECUSADO.**
- Publica no RabbitMQ os eventos pagamento.aprovado ou pagamento.recusado.

MS Entrega

- Consome pagamento.aprovado, simula a emissão da Nota Fiscal e despacho do produto, e publica pedido.enviado.

MS Promoções (valor 0,4)

- **(0,1) Consome o evento interesse.promocao.**
- **(0,3) Gera as promoções e as notificações destas aos clientes interessados. Para isso, ele identifica os interesses e e-mails dos consumidores cadastrados e integra-se com uma API Externa de e-mail (ex: *Resend*) para disparar as notificações de promoções.**

Mock de Pagamento (Sistema Externo) (Valor: 0,2)

- **Aplicação web externa simples que simula um provedor de pagamentos.**
- **(0,1) Processa a solicitação de cobrança do MS Pagamento via REST.**
- **(0,1) Abre uma página no navegador onde o usuário clica nos botões "Pagamento Aprovado" ou "Pagamento Recusado". Quando o usuário clicar, faz uma chamada HTTP POST (*Webhook*) para a URL informada pelo MS Pagamento.**

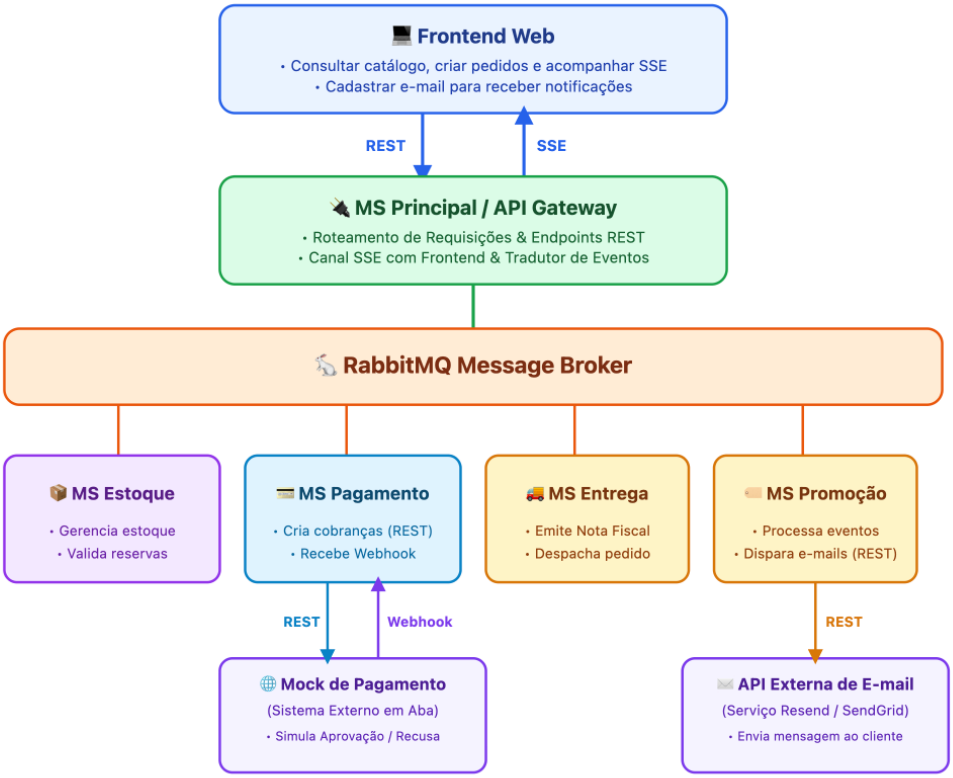


Figura 1 - Arquitetura

Observações:

- Desenvolva uma **interface gráfica** com recursos de interação apropriados.
- É obrigatória a defesa da aplicação para obter a nota.
- O desenvolvimento do sistema deve ser em dupla.